

# E-Commerce System

---

## Course Completion Project

**Stack:** Node.js · Express · TypeORM · Angular

**Database:** MySQL or PostgreSQL

### CONTENTS

1. [Project Overview](#)
2. [Product Taxonomy](#)
3. [User Roles](#)
4. [Security Requirements](#)
5. [Authentication & Session Management](#)
6. [Guest Features](#)
7. [Customer Features](#)
8. [Search & Filtering](#)
9. [Cart & Checkout](#)
10. [Orders](#)
11. [Image Handling](#)
12. [Admin Features](#)
13. [Frontend & UI Requirements](#)
14. [Build & Serving](#)
15. [Deliverables](#)
16. [Grading Rubric](#)

---

## 1. Project Overview

You are required to design and build a fully functional e-commerce web application. The system must support three levels of access — Guest, Customer, and Admin — and provide a complete shopping experience: browsing, searching, filtering, cart management, checkout, and order history.

The backend must be built with Node.js and Express using TypeORM to communicate with a relational database. The frontend must be built with Angular. Both must be integrated so that a single Node.js process serves the entire application.

## 2. Product Taxonomy

Products are organised in a four-level hierarchy:

- **Type** — the broadest grouping (e.g., Electronics, Stationery, Furniture)
- **Category** — belongs to a Type (e.g., Computer Peripherals)
- **Sub-Category** — belongs to a Category (e.g., Keyboards)
- **Product** — belongs to a Sub-Category (e.g., Anker Multimedia Keyboard)

Every product must belong to exactly one Sub-Category. Every Sub-Category belongs to exactly one Category, and every Category belongs to exactly one Type.

## 3. User Roles

| Role     | Description                                                                                                                                                         |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Guest    | Can browse the homepage, view product listings, view product detail pages, and use search and filters. Cannot add to cart, checkout, or access any account feature. |
| Customer | All guest capabilities, plus cart, checkout, order history, profile management, and password management.                                                            |
| Admin    | Access to an admin panel within the same application. Can manage products, view all orders, and manage customer accounts.                                           |

## 4. Security Requirements

**Important:** Implementing proper security is mandatory. The following security practices (covered during the course) must be applied throughout the project:

- **Password Security:** Use a strong, slow hashing algorithm ( `bcrypt` recommended) with a proper salt. Never store passwords in plain text.
- **SQL Injection Prevention:** All database queries must use TypeORM's query builder or parameterized queries. Raw SQL with string concatenation is not acceptable.
- **Input Validation & Sanitization:** Validate and sanitize all user inputs on both backend and frontend (especially product data, registration fields, search terms, etc.).
- **Authorization & Role-Based Access Control (RBAC):** Every protected route and API endpoint must verify the user's role and session. Admin routes must be completely inaccessible to Customers and Guests.

- **Session & Cookie Security:**
  - Use HTTP-only and Secure cookies for the authentication token.
  - Implement proper session validation on every protected request using the in-memory session store.
- **Account Locking:** When an Admin locks a customer account, the lock must take effect immediately (next request must fail and log the user out).
- **Rate Limiting** (Recommended): Implement basic rate limiting on login and registration endpoints to mitigate brute-force attacks.
- **CORS Configuration:** Properly configure CORS to allow only your Angular frontend origin.
- **Error Handling:** Never expose sensitive information (stack traces, database errors, internal paths, etc.) in error responses shown to the user.
- Any other measures you can think of.

Failure to implement these security measures will result in significant mark deductions.

---

## 5. Authentication & Session Management

---

### 5.1 Registration

Any visitor may register as a Customer by providing their name, email address, and a password. Passwords must be stored securely using a hashing algorithm — plain-text storage is not acceptable.

### 5.2 Login

Registered users may log in with their email and password. Upon successful login, a token must be issued and stored in an HTTP-only cookie. Do not store the token in `localStorage` or `sessionStorage`.

Every active login must also create an entry in a server-side session store backed by a plain Node.js `Map` object held in memory. Both the cookie and the session entry must be checked on every protected request.

### 5.3 Account Locking

An Admin may lock any Customer account. The lock must take effect immediately — if the Customer is currently logged in, their next request must be rejected and they must be logged out. There must be no delay; the session store is the mechanism that enforces this in real time.

### 5.4 Logout

Logging out must clear the cookie and remove the session entry from the in-memory store.

### 5.5 Forgot Password

A user who has forgotten their password may request a reset. The system must generate a temporary numeric code and display it directly on the screen (no email delivery is required — this is a mock). The user enters the code and is then permitted to set a new password. Codes must expire after a short time.

## 5.6 Role Enforcement

All routes and Angular pages must enforce role requirements. Admin pages must be inaccessible to Customers and Guests. Customer-only pages must redirect unauthenticated Guests to the login page.

---

## 6. Guest Features

---

- View the homepage with featured or all products displayed with images.
  - Browse products by Type, Category, or Sub-Category.
  - Search for products by keyword.
  - Filter search or browse results by category, price range, or any other criteria the student considers appropriate.
  - View individual product detail pages.
  - Use the share button on a product detail page.
  - Register for an account.
  - Log in to an existing account.
  - Request a password reset via the forgot-password flow.
- 

## 7. Customer Features

---

- All guest browsing and search features.
  - Add products to a persistent cart.
  - View, update quantities, and remove items from the cart.
  - Proceed through checkout and place an order.
  - View a payment confirmation page after a successful order.
  - View a list of past orders.
  - View the full details of any past order, including items, quantities, prices, and payment method.
  - Edit profile information (name, email).
  - Change password after login (requires current password).
  - Log out.
- 

### 8.1 Full-Text Search

The search system must query across product names and descriptions. It must return results from any level of the taxonomy — a keyword may match products in multiple Types, Categories, or Sub-Categories simultaneously, and all matches must be returned.

For example, if a user searches for *table*, the results must include products such as a *Wooden Table* listed under Furniture as well as a *Multiplication Table Book* listed under Stationery > Kids > Textbooks. Results from different parts of the taxonomy must all appear together in the same result set.

## 8.2 Browsing by Taxonomy

If a user selects or enters a Sub-Category name such as *Keyboards*, all products belonging to that Sub-Category must be displayed. Results must be paginated — the number of results per page is the student's choice, but must be consistent and configurable.

## 8.3 Filters

The following filters must be available on any product listing or search results page:

- Filter by Type, Category, or Sub-Category
- Filter by price range (minimum and maximum)
- Any additional filter the student considers useful (e.g., in-stock only)

Filters may be combined. All filtering must happen server-side; the API must accept filter parameters and apply them to the query.

---

# 9. Cart & Checkout

## 9.1 Cart

Only logged-in Customers may use the cart. The cart must persist — if a Customer adds items, logs out, and logs back in, the cart contents must still be present.

The Customer must be able to adjust quantities and remove items before proceeding to checkout.

## 9.2 Payment Methods

No real payment gateway is required. During checkout the Customer must be shown a selection of payment methods and must choose one before confirming the order. Examples of methods to offer: Credit Card, Debit Card, Cash on Delivery, Bank Transfer. The chosen method must be stored with the order.

## 9.3 Order Confirmation

After a successful order is placed, the Customer must be redirected to a payment confirmation page that summarises the order: items purchased, total amount, and payment method selected. The cart must be cleared after this point.

---

## 10. Orders

---

Each order must record the Customer it belongs to, the date it was placed, the payment method, the total amount, and the individual items with their quantities and the price at the time of purchase (not the current price, which may change later).

Customers can view a list of all their past orders and drill into any order to see its full details.

---

## 11. Image Handling

---

**Instructor Note:** Use `express.static` to serve images directly from a folder named `ProductImages`. Store only the relative file path of each image in the database — do not store image data in the database itself. Not all products will have images; in those cases a default placeholder image must be shown automatically.

- Product images are stored as files in a dedicated `ProductImages` folder on the server.
  - The database stores only the relative path to the image file.
  - Images are served directly by Express using `express.static`.
  - If a product has no image, a default image is displayed in its place.
  - The Admin can upload an image when adding or editing a product.
- 

## 12. Admin Features

---

The Admin panel is part of the same Angular application, accessible under a separate route (e.g., `/admin`). It must be fully protected — no Customer or Guest may access any admin route or admin API endpoint.

### 12.1 Product Management

- View a list of all products.
- Add a new product, including its name, description, price, stock quantity, taxonomy placement (Type → Category → Sub-Category), and an optional image.
- Edit any existing product's details and image.
- Delete a product.

### 12.2 Customer Management

- View a list of all registered Customers.

- Lock or unlock any Customer account. The lock must take effect immediately, even if the Customer is currently logged in.

## 12.3 Order Management

- View all orders placed by all Customers.
  - View the full details of any order.
- 

## 13. Frontend & UI Requirements

---

### 13.1 General

- The application must use Angular with Angular Router for navigation.
- The Admin panel must be implemented as a separate lazy-loaded Angular module.
- Route guards must protect Customer and Admin routes appropriately.
- The application must handle and display meaningful error messages (e.g., incorrect password, out-of-stock, account locked).

### 13.2 Homepage

The homepage must be visually appealing and display products with their images. It is the primary landing page for all visitors and must make a good first impression. A default image must be shown for products without an uploaded image.

### 13.3 Product Detail Page

Each product must have its own detail page showing all product information, its image (or default), its full taxonomy path, and an Add to Cart button (visible only to logged-in Customers). A share button must be present that allows the user to share the product page URL. If the server is running and the URL is pasted on another device, the user should be directed to the same product page. Product details are visible to everyone; only logged-in Customers can add to cart or place orders.

### 13.4 Styling

- Use plain CSS only — no CSS frameworks, no Tailwind, no Bootstrap.
  - The design must be clean, readable, and consistent across all pages.
  - The homepage must stand out with a well-structured layout and clear product imagery.
- 

## 14. Build & Serving

---

The Angular project must be built using `ng build`. The build output must be served by the same Express server that provides the API. This means a visitor who opens the application in a browser must be hitting a single Node.js server for both the frontend and all API calls.

## 15. Deliverables

| # | Item                            | Notes                                                                                                                                                                        |
|---|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | SQL file                        | Optional. Include any stored procedures, triggers, or seed data scripts that cannot be expressed in application code. Not required if everything is handled through TypeORM. |
| 2 | Backend + Frontend source files | Submit all source files. The <code>node_modules</code> directory must be excluded.                                                                                           |
| 3 | Angular build                   | The Angular code without <code>node_modules</code> directory.                                                                                                                |

### Final Deliverable

A single compressed zip file containing all of the above.

## 16. Grading Rubric

| Area                                                                                                                                                                         | Marks |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| Database design — correct relationships, normalisation, and appropriate use of constraints                                                                                   | 10    |
| Security implementation — password hashing, input validation & sanitization, SQL injection prevention, proper authorization/RBAC, secure cookies, CORS, error handling, etc. | 15    |
| Authentication — registration, login, logout, cookie-based token, in-memory session, immediate account lock enforcement                                                      | 10    |
| Forgot password mock flow (code displayed, expiry enforced, password reset successful)                                                                                       | 5     |
| Product taxonomy — correct four-level hierarchy, browsing by level                                                                                                           | 5     |
| Full-text search — cross-taxonomy results, correctness of results (e.g., "table" example)                                                                                    | 8     |
| Filtering and pagination on listings and search results                                                                                                                      | 8     |
| Cart — persistence, quantity management, correct clearing after order                                                                                                        | 8     |
| Checkout — payment method selection, order creation, confirmation page                                                                                                       | 7     |



|                                                                                      |            |
|--------------------------------------------------------------------------------------|------------|
| Order history — list and detail view, correct price snapshot                         | 5          |
| Image handling — static serving, default fallback, admin upload                      | 5          |
| Admin panel — product CRUD, customer locking, order viewing                          | 8          |
| Angular — routing, lazy loading, guards, share button, edit profile, change password | 6          |
| Homepage design quality and overall UI consistency (plain CSS only)                  | 5          |
| <b>Total</b>                                                                         | <b>100</b> |

---

This document defines the minimum requirements. Students are encouraged to think carefully about edge cases, data integrity, security, and user experience beyond what is explicitly stated. Additional thoughtful features or excellent security practices may be considered positively during evaluation.